

# OATS: Online Data Augmentation for Time Series Foundation Models

Junwei Deng  
University of Illinois  
Urbana-Champaign

Jiaqi W. Ma  
University of Illinois  
Urbana-Champaign

Ming Jin  
Griffith University

Chenghao Liu  
Datadog

Xu Zhang  
Microsoft

Li Zhao  
Microsoft

Jiang Bian  
Microsoft

## Abstract

Time Series Foundation Models (TSFMs) are a powerful paradigm for time series analysis and are often enhanced by synthetic data augmentation to improve the training data quality. Existing augmentation methods, however, typically rely on heuristics and static paradigms. Motivated by dynamic data optimization, which shows that the contribution of samples varies across training stages, we propose OATS (Online Data Augmentation for Time Series Foundation Models), a principled strategy that generates synthetic data tailored to different training steps. OATS leverages valuable training samples as principled guiding signals and dynamically generates high-quality synthetic data conditioned on them. We further design a diffusion-based framework to produce realistic time series. Experiments on TSFMs demonstrate that OATS consistently outperforms regular training and yields substantial performance gains over static data augmentation baselines across six validation datasets and two TSFM architectures. The code is available at the link <https://github.com/microsoft/TimeCraft>.

## CCS Concepts

• Computing methodologies → Machine learning.

## Keywords

time series foundation models, data augmentation, online data selection, data attribution, diffusion models, time series forecasting

## 1 Introduction

Time series modeling plays a critical role across a wide range of domains, including finance [11, 15], healthcare [8], climate science [16], and industrial monitoring [37]. Recent developments in time series foundation models (TSFMs) have further advanced this field by leveraging large-scale datasets collected from multiple third-party sources [2, 35], enabling cross-domain learning and zero-shot generalization. Nevertheless, the success of TSFMs relies on the availability of high-quality data. Typical challenges in time series datasets include missing values [10], heterogeneous sampling rates [31], imbalanced domain distributions [35], data duplication [17]. These issues make it more difficult to curate reliable large-scale time series datasets than in domains such as natural language processing [6, 22]. As the community moves toward large-scale foundation models, synthetic data augmentation has emerged as a critical and widely adopted method for enhancing training

data with synthetic samples. Leveraging the high controllability of time series patterns and operational simplicity, synthetic data augmentation can not only address data scarcity but also enriches domain diversity and improves the robustness of TSFMs [18].

Various data augmentation methods have been proposed in TSFM studies to enrich training datasets with realistic and diverse synthetic data. These approaches can be broadly categorized into two groups. The first involves directly introducing *manually designed patterns* to synthesize data. Such patterns include sinusoidal waves [7], decomposed time series [5, 14], and handcrafted kernel banks [2, 24, 33]. The second group focuses on applying *basic transformations to existing time series data* to generate synthetic variants. Examples of such techniques include smoothing, jittering [19, 27], and TSMixup [1, 2, 19].

While these approaches have shown empirical effectiveness, they rely on *handcrafted heuristics* that are often *agnostic to the model training process*, which result in two critical challenges in designing high-performance data augmentation strategies. First, the quality of time series data for TSFMs is difficult to quantify in a principled way. The heuristics that work for one time series model may fail for another [13, 18]. Second, recent studies show that the value of the same data sample varies across the training process [28], which challenges the capacity of the static data augmentation paradigm that generates synthetic data once and uniformly incorporated along the whole training process.

In this study, we address these challenges by leveraging recent advancements in *data attribution* [3, 12] and *online data optimization* [30]. For the first challenge, data attribution aims to quantify the influence of individual training data points on model outputs; instead of studying heuristic rules like which pattern of time series data is helpful for the training, data attribution allows us to define and assess the quality of time series data by its influence on a utility function, e.g., validation loss, in a principle way. For the second challenge, we go beyond the static data augmentation paradigm in existing TSFM literature and conduct online data augmentation, which incorporates the training process information and dynamically generates high-quality data for each training step. For example, our analysis of a TSFM training run (Section 4.4) reveals that the contribution of each training sub-dataset evolves dynamically across training steps and does not align with its proportion in the corpus, suggesting that a static augmentation strategy is unlikely to remain optimal throughout training.

Concretely, we propose OATS (Online Data Augmentation for Time Series Foundation Models), a strategy to dynamically generate high-quality synthetic data in a principled manner. OATS generates high-quality synthetic data by using training samples with high data attribution scores [12] as guiding signals. OATS consists of two core components: **Time-series influence scores (TSIS)** integrate data attribution with time series-specific knowledge to dynamically assess the quality of each training sample in a principled manner to create a generation guiding signal. **High-quality guided data augmentation** leverages the guiding signal to condition a diffusion model trained on a small subset of the TSFM training data for synthetic data generation.

We summarize our contributions in this paper as follows:

- **An Online Data Augmentation Paradigm for TSFM.** We propose a new paradigm for TSFM training that generates synthetic data tailored to each training step. This approach expands the potential of the traditional static data augmentation paradigm by taking the training process information into consideration.
- **A Novel Online Data Augmentation Strategy: OATS.** OATS leverages valuable training samples identified by data attribution scores as guiding signals and employs a diffusion model to generate synthetic data conditioned on these signals for each training step. Overall, OATS offers a principled approach to dynamically generating high-quality synthetic data.
- **Comprehensive Empirical Evaluation.** We evaluate OATS on six forecasting datasets across two TSFM architectures. Through systematic ablations, we isolate the contributions of TSIS guidance, the generation strategy, and the validation set design, demonstrating that each component contributes to the overall improvement over baseline augmentation methods.

## 2 Related Works

*Time Series Data Augmentation.* Traditional data augmentation for time series mostly relies on static, manually designed heuristics. Techniques such as injecting sinusoidal wave [7], transformations of decomposed time series [5], jittering [27], and TSMixup [2] aim to enrich datasets before training. More advanced, AutoTCL [38] leverages parameterized contrastive learning to decompose time series into information and noise components, applying transformations to the former. However, these paradigms are executed offline, remaining agnostic to the dynamic optimization state of the TSFMs during training. Notably, although Taga et al. [26] proposes an online algorithm combining RHO-based selection with traditional transformations, its simplistic generation process may lead to performance degradation. Moreover, the RHO-based approach incurs substantial computational overhead as it requires training a separate reference model.

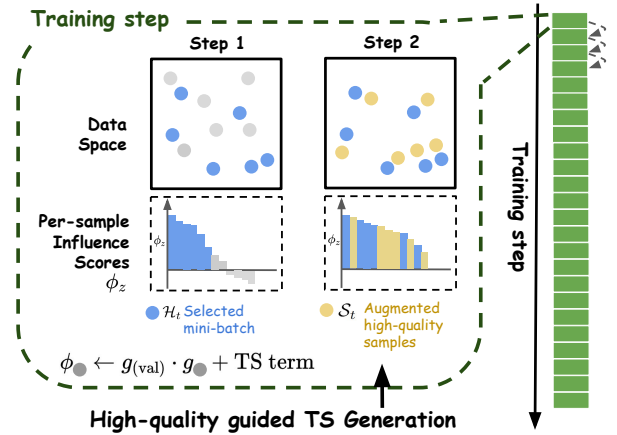
*Dynamic Data Optimization and Online Selection.* Recent advances highlight that the contribution of training data changes significantly across optimization stages. In the large language models, DoReMi [34] demonstrates the necessity of dynamic data mixing. Down to the instance level, GREATs [30] formulates online batch selection as an optimization problem, leveraging Taylor expansions to evaluate data influence efficiently. Despite these strides, existing online methods mainly focus on sample pruning or reweighting

coupled with naive static augmentations. OATS pioneers the integration of rigorous online influence evaluation with a high-fidelity conditional generation, effectively closing the loop between real-time data valuation and dynamic synthetic data generation.

*Time Series Generation.* Time series generation has been explored using a variety of model architectures. GAN-based approaches train adversarial objectives to generate realistic sequences [36]. VAE-based methods design specialized decoders tailored for time series [4]. More recently, diffusion-based methods have been developed, leveraging different denoising architectures. Beyond unconditional synthesis, conditional generation has been studied in settings such as metadata-conditioned generation [20] and prompt-based generation [9].

## 3 Method

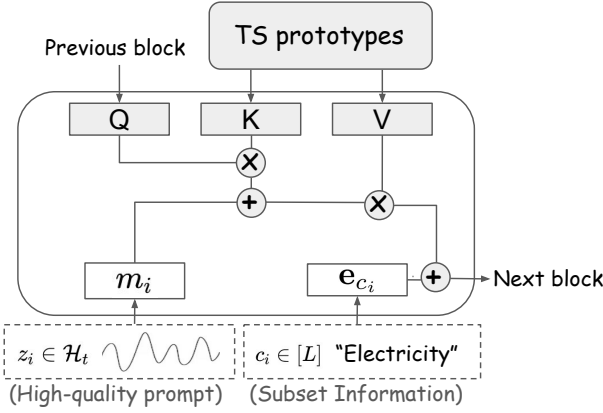
In this section, we introduce the modules of Online Data Augmentation for Time Series Foundation Models (OATS). OATS consists of two modules, i.e., *Time-series Influence Scores (TSIS)* (Section 3.2) quantitatively estimate the quality of time series samples and to identify valuable data as guiding signal; *High-quality guided Data Generation* (Section 3.3) generates synthetic samples conditioned on the guiding signal. A diagram of the whole algorithm is presented in Figure 1, and an algorithm block is shown in Section 3.4.



**Figure 1: Architecture of OATS.** OATS employs two modules: ① Time-Series Influence Scores (TSIS) create generation guiding signals as high-quality data samples, and ② guide time series synthetic data generation for augmentation.

### 3.1 Problem Setup

*Set-up of online data augmentation in TSFM.* Suppose we have a training dataset  $\mathcal{D}_{Tr}$  of size  $N$ . We also have a validation dataset  $\mathcal{D}_{val} = \{z_v | v = [N_v]\}$ ,  $z_v \in \mathcal{Z}$  and  $\mathcal{D}_{val}$  is not overlapped with  $\mathcal{D}_{Tr}$ .



**Figure 2: The architecture of the denoising diffusion model to generate synthetic data conditioned on constructed generation signals.**

A TSFM parameterized by  $w \in \mathcal{W}$  is being trained on  $\mathcal{D}_{tr}$  to minimize the loss function  $\ell$  via an iterative optimization algorithm, e.g., stochastic gradient descent [23] for  $T$  steps. The intermediate checkpoints of each step are represented as  $\{w_t | t = [T]\}$ . At iteration  $t$ , a mini-batch of data  $\mathcal{B}_t = \{z_1, z_2, \dots, z_B\}$  is sampled. Online data augmentation then generates synthetic samples  $\mathcal{S}_t$ , which are combined with all or part of  $\mathcal{B}_t$  to update the model.

### 3.2 Construct Guiding Signal via Time Series Influence Scores

Generating synthetic data is widely used in the training process, yet a core question remains unsettled: *What should be generated?* A consistent and universally accepted answer has yet to emerge for the definition of the quality criteria for time series synthetic data in data augmentation. In OATS, we employ *data attribution* [12, 21] as a principled method to estimate the influence of a data point on the model output, which has shown significant usefulness in dataset optimization in different areas. The data attribution scores to validation loss are taken as the quantitative quality indicator.

We design a time-series influence score (TSIS)<sup>1</sup> function with respect to a validation dataset  $\mathcal{D}_{val}$  as  $\mathcal{F}_{\mathcal{D}_{val}}$  for each sample to be  $\mathcal{F}_{\mathcal{D}_{val}} : \mathcal{D}_{tr} \times \mathcal{W} \rightarrow \mathbb{R}$ , which estimates the data attribution score of a data sample to be trained on  $w_t$  with respect to the performance on  $\mathcal{D}_{val}$ . A larger score indicates that training on  $z$  for the next training step ( $t + 1$ ) leads to better performance on  $\mathcal{D}_{val}$ . The TSIS function can be defined as:

$$\mathcal{F}_{\mathcal{D}_{val}}(z_i, w_t) = \underbrace{g_{(val);t}}_{\text{Influence Score}} \cdot \underbrace{g_{z_i;t} - \mathbf{I}_{\text{SNR}(z_i) < k}}_{\text{TS-specific quality}} \cdot \infty, \quad (1)$$

where  $g_{(val);t} = \nabla_w \sum_{v_i \in \mathcal{D}_{val}} \ell(v_i, w_t) / |\mathcal{D}_{val}|$  and  $g_{z_i;t} = \nabla_w \ell(z_i, w_t)$ . The data attribution score is a first-order Taylor approximation of the validation loss change, where a careful derivation is stated in Lemma 1. We additionally consider TS-specific quality indicators, which we use signal-to-noise ratio (SNR) in a pre-selection process, where  $k$  is a threshold of minimal SNR and  $\mathbf{I}$  is the indicator

<sup>1</sup>We will use “influence scores” and “data attribution scores” interchangeably.

function. High-quality data  $\mathcal{H}_t$  is identified as the samples with top- $k$  TSIS scores in each mini-batch  $\mathcal{B}_t$  of training step  $t$ , which will be used as guiding signals for synthetic data generation.  $k$  is a hyperparameter while we follow a previously-adopted ratio (50%) in Wang et al. [30].

**LEMMA 1 (FIRST-ORDER TAYLOR APPROXIMATED INFLUENCE SCORE).** *The influence score term of TSIS is a first-order Taylor approximation of the difference of a utility function before and after a training step. Typically, we define the utility function to be the validation loss, which can be represented as*

$$\ell(\mathcal{D}_{val}, w_t) = \frac{1}{|\mathcal{D}_{val}|} \sum_{v_i \in \mathcal{D}_{val}} \ell(w_t, v_i)$$

$$\begin{aligned} \ell(\mathcal{D}_{val}, w_t) - \ell(\mathcal{D}_{val}, w_{t+1}) &= (w_t - w_{t+1})^\top \nabla_w \ell(\mathcal{D}_{val}, w_t) \\ &\quad + O(\|w_{t+1} - w_t\|^2) \\ &\simeq (w_t - w_{t+1})^\top \nabla_w \ell(\mathcal{D}_{val}, w_t) \\ &= \eta_t \nabla_w \ell(z_i, w_t)^\top \nabla_w \ell(\mathcal{D}_{val}, w_t), \end{aligned}$$

where the model is updated through gradient descent<sup>2</sup> on a data sample  $z_i$  with learning rate  $\eta_t$ , i.e.,

$$w_{t+1} = w_t - \eta_t \nabla_w \ell(z_i, w_t).$$

Given that the learning rate  $\eta_t$  is typically small, the error of first-order Taylor approximation with level  $O(\|w_{t+1} - w_t\|^2)$ , or  $O(\eta^2)$  if the norm of gradient of loss is bounded, is small enough to provide an accurate estimate.

### 3.3 High-quality Data Guided Data Generation

Once the guiding signal is constructed, a natural question arises: *how can the synthetic data be generated?* We design a high-quality data guided generation model  $\mathcal{G}$  for online synthetic data generation. We aim to model the conditional distribution  $p(\hat{z} | \mathcal{H}_t)$ , which defines the probability of generating a sample  $\hat{z}$  given the guiding signal  $\mathcal{H}_t \subseteq \mathcal{B}_t$  for each training step  $t$ . The conditional generation model  $\mathcal{G}$  serves as a parameterized approximation of this distribution, enabling practical sampling via  $\hat{z} \sim \mathcal{G}(\mathcal{H}_t)$ . We design the architecture of  $\mathcal{G}$  to be a diffusion model utilizing a time series semantic prototype module and take  $\mathcal{H}_t$  as a generation condition. To incorporate the condition into the intermediate layers of the noise prediction network, we take Huang et al. [9] as our backbone model.

Our generation model  $\mathcal{G}$  introduces a set of latent arrays  $P$  as time series prototypes and extracts prototype weights  $\mathcal{M} = \{m_i | z_i \in \mathcal{H}_t\}$ , where we leverage the same design of weight extractor and time series prototypes initialization in Huang et al. [9]. We additionally include a class conditional guidance based on the subset information to enhance the condition, denoted as  $\mathcal{C}_{high} = \{c_i | z_i \in \mathcal{H}_t\}$ , where each  $c_i \in [L]$  indicates which disjoint subset  $\mathcal{D}_l$  the sample  $z_i$  belongs to.

<sup>2</sup>Here we use SGD on a single data sample, which has been widely accepted as a reasonable approximation for other optimizers like Adam in data attribution studies[29, 30].

**Table 1: Performance comparison of various data augmentation methods on encoder-only TSFM with prediction length be 192 / average over all prediction lengths) and  $\epsilon = 1$ . Bold means the best result. Light green background means that the performance is better than regular training process, while light red background means that the performance is worse than regular training process. The error bar shows the standard error of mean over 5 independent runs.**

| Dataset     | Pred. length | OATS                                |                                     | TSMixup           |                                     | Jitter            |                                     | Regular           |                   |
|-------------|--------------|-------------------------------------|-------------------------------------|-------------------|-------------------------------------|-------------------|-------------------------------------|-------------------|-------------------|
|             |              | NLL                                 | MAPE                                | NLL               | MAPE                                | NLL               | MAPE                                | NLL               | MAPE              |
| ETTh1       | 192          | <b>1.627 <math>\pm</math> 0.042</b> | <b>0.672 <math>\pm</math> 0.043</b> | 1.725 $\pm$ 0.031 | 0.759 $\pm$ 0.038                   | 1.715 $\pm$ 0.047 | 0.691 $\pm$ 0.044                   | 1.870 $\pm$ 0.019 | 0.844 $\pm$ 0.056 |
|             | Overall      | <b>1.614 <math>\pm</math> 0.020</b> | <b>0.623 <math>\pm</math> 0.024</b> | 1.715 $\pm$ 0.018 | 0.700 $\pm$ 0.019                   | 1.731 $\pm$ 0.025 | 0.650 $\pm$ 0.019                   | 1.854 $\pm$ 0.016 | 0.783 $\pm$ 0.037 |
| ETTh2       | 192          | <b>1.872 <math>\pm</math> 0.014</b> | <b>0.208 <math>\pm</math> 0.005</b> | 2.083 $\pm$ 0.016 | 0.271 $\pm$ 0.006                   | 2.068 $\pm$ 0.020 | 0.256 $\pm$ 0.007                   | 2.118 $\pm$ 0.028 | 0.267 $\pm$ 0.009 |
|             | Overall      | <b>1.863 <math>\pm</math> 0.013</b> | <b>0.222 <math>\pm</math> 0.004</b> | 2.097 $\pm$ 0.011 | 0.292 $\pm$ 0.003                   | 2.072 $\pm$ 0.009 | 0.273 $\pm$ 0.005                   | 2.128 $\pm$ 0.014 | 0.290 $\pm$ 0.008 |
| ETTh1       | 192          | <b>1.794 <math>\pm</math> 0.035</b> | 0.681 $\pm$ 0.025                   | 1.883 $\pm$ 0.030 | 0.634 $\pm$ 0.025                   | 1.959 $\pm$ 0.028 | <b>0.629 <math>\pm</math> 0.035</b> | 1.897 $\pm$ 0.040 | 0.758 $\pm$ 0.059 |
|             | Overall      | <b>1.814 <math>\pm</math> 0.020</b> | 0.709 $\pm$ 0.021                   | 1.860 $\pm$ 0.027 | <b>0.634 <math>\pm</math> 0.011</b> | 1.913 $\pm$ 0.026 | 0.644 $\pm$ 0.021                   | 1.915 $\pm$ 0.019 | 0.812 $\pm$ 0.031 |
| ETTh2       | 192          | <b>1.857 <math>\pm</math> 0.063</b> | <b>0.267 <math>\pm</math> 0.007</b> | 2.123 $\pm$ 0.029 | 0.294 $\pm$ 0.011                   | 2.127 $\pm$ 0.022 | 0.296 $\pm$ 0.013                   | 2.084 $\pm$ 0.047 | 0.282 $\pm$ 0.015 |
|             | Overall      | <b>1.876 <math>\pm</math> 0.025</b> | <b>0.263 <math>\pm</math> 0.005</b> | 2.113 $\pm$ 0.020 | 0.294 $\pm$ 0.005                   | 2.122 $\pm$ 0.022 | 0.298 $\pm$ 0.005                   | 2.086 $\pm$ 0.021 | 0.283 $\pm$ 0.008 |
| Weather     | 192          | <b>3.193 <math>\pm</math> 0.043</b> | <b>1.778 <math>\pm</math> 0.127</b> | 3.413 $\pm$ 0.041 | 2.016 $\pm$ 0.040                   | 3.460 $\pm$ 0.043 | 2.619 $\pm$ 0.280                   | 3.486 $\pm$ 0.051 | 2.331 $\pm$ 0.285 |
|             | Overall      | <b>3.216 <math>\pm</math> 0.036</b> | <b>1.659 <math>\pm</math> 0.069</b> | 3.428 $\pm$ 0.023 | 1.948 $\pm$ 0.033                   | 3.499 $\pm$ 0.020 | 2.591 $\pm$ 0.145                   | 3.526 $\pm$ 0.026 | 2.267 $\pm$ 0.125 |
| Electricity | 192          | <b>5.967 <math>\pm</math> 0.021</b> | <b>0.515 <math>\pm</math> 0.071</b> | 5.983 $\pm$ 0.010 | 0.553 $\pm$ 0.034                   | 6.050 $\pm$ 0.015 | 0.659 $\pm$ 0.062                   | 6.266 $\pm$ 0.019 | 0.750 $\pm$ 0.033 |
|             | Overall      | <b>5.945 <math>\pm</math> 0.011</b> | <b>0.507 <math>\pm</math> 0.045</b> | 5.987 $\pm$ 0.007 | 0.555 $\pm$ 0.013                   | 6.044 $\pm$ 0.010 | 0.642 $\pm$ 0.025                   | 6.260 $\pm$ 0.015 | 0.767 $\pm$ 0.021 |

The generation (reverse diffusion) for each prompt can be expressed as

$$p(z^{(t-1)}|z^{(t)}, \mathcal{D}_{\text{high}}) = \mathcal{N}(z^{(t-1)}|\mu(z^{(t)}, t, \underbrace{m_i, c_i}_{\text{High-quality data guidance}}, \sigma_t), \quad (2)$$

where  $z^{(t)}$  is the intermediate sample at diffusion timestep  $t$ ,  $\mathcal{N}$  is Gaussian distribution,  $\sigma_t$  is the noise covariance at diffusion step  $t$ . The architecture of the parameterized conditional noise predictor incorporates the prototype weights  $m_i$  and class condition  $c_i$ . We present the architecture of the high-quality guided data augmentation in Figure 2 in an intermediate  $u^{\text{th}}$  U-Net layer. The formula can be represented as

$$h^{(u)} = \text{FF}(\text{Softmax}(\frac{QK^T}{\sqrt{d}} + m_i)V||e_{c_i}), \quad (3)$$

where  $Q = h^{(u-1)}W_Q^{(u)}$ ,  $K = PW_K^{(u)}$  and  $V = PW_V^{(u)}$  are the query, key, and value for the attention.  $W_Q \in \mathbb{R}^{d \times d}$ ,  $W_K \in \mathbb{R}^{d \times d}$ ,  $W_V \in \mathbb{R}^{d \times d}$  are learnable parameters,  $P \in \mathbb{R}^{N_p \times d}$  is the prototype latent arrays,  $e_{c_i} \in \mathbb{R}^{d_c}$  is the embedding of class  $c_i$ ,  $d$  and  $d_c$  is the hidden dimension of prototype embedding and class embedding, and  $\text{FF}$  is a feed-forward network. The final U-Net layer's output  $h^{\text{final}}$  is followed by another feed-forward network to produce the predicted noise.

### 3.4 Algorithm Overview

We summarize Online Data Augmentation for Time Series Foundation Models (OATS) and present in Algorithm 1. To further improve efficacy and efficiency, we propose an explore-exploit mechanism that leverages cached influence scores to reduce computational overhead in Appendix A and discuss the computational cost in Appendix B.

#### Algorithm 1 Online Data Augmentation for Time Series Foundation Models (OATS)

**Require:** Training dataset  $\mathcal{D}_{tr}$ , validation dataset  $\mathcal{D}_{val}$ , training batch size  $b$ , conditional augmentation algorithm  $\mathcal{G}$ , training loss  $\ell$ , TSIS function  $\mathcal{F}_{\mathcal{D}_{val}}$ .  
Initialize model  $w_0$   
Initialize TSIS per subset  $\Phi_{\mathcal{D}_l}$ ,  $l \in [L]$  to be subset proportion.  
**for**  $t = 1$  to  $T$  **do**  
    Sample  $\mathcal{B}_t$  ▷ Step 1  
    Calculate  $\mathcal{F}_{\mathcal{D}_{val}}(z_i, w_t)$  for  $z_i \in \mathcal{B}_t$   
    Update  $\Phi_{\mathcal{D}_l}$  according to Equation 4.  
    Select a subset  $\mathcal{H}_t \subseteq \mathcal{B}_t$  of samples with top- $b/2$  of value  $\mathcal{F}_{\mathcal{D}_{val}}(z_i, w_t)$ .  
    Generate  $b/2$  samples as  $\mathcal{S}_t$  using  $\mathcal{G}$  guided by  $\mathcal{H}_t$ . ▷ Step 2  
    Update  $w_t$  on mini-batch data  $\mathcal{H}_t \cup \mathcal{S}_t$  and get  $w_{t+1}$ ,  
    continue to next step  $t + 1$ .  
**end for**

## 4 Experiments

In this section, we present the empirical evaluation of OATS. We first introduce the experiment setup in Section 4.1. We then evaluate the performance of OATS on typical TSFM architectures and datasets in Section 4.2. We also include several ablation studies in Section 4.3.

### 4.1 Experiment Setup

**Model.** We conduct experiments on two typical TSFM architectures, i.e., encoder-only and decoder-only Transformer. We follow the same training settings and model definitions, which incorporate patch embedding, rotary positional embedding, and a mixture of distributions to better adapt to time series forecasting while preserving extensibility, in Yao et al. [35] to match the popular TSFM forecasting models, Moirai [31] and Chronos [2]. We will call these



**Table 2: Performance comparison of various data augmentation methods on decoder-only TSFM with prediction length be 192 / average over all prediction lengths) and  $\epsilon = 1$ . Bold means the best result. Light green background means that the performance is better than regular training process, while light red background means that the performance is worse than regular training process. The error bar shows the standard error of mean over 5 independent runs.**

| Dataset     | Pred. length | OATS                                |                                     | TSMixup           |                   | Jitter            |                                     | Regular           |                   |
|-------------|--------------|-------------------------------------|-------------------------------------|-------------------|-------------------|-------------------|-------------------------------------|-------------------|-------------------|
|             |              | NLL                                 | MAPE                                | NLL               | MAPE              | NLL               | MAPE                                | NLL               | MAPE              |
| ETTh1       | 192          | <b>1.654 <math>\pm</math> 0.018</b> | <b>0.659 <math>\pm</math> 0.027</b> | 1.691 $\pm$ 0.013 | 0.664 $\pm$ 0.027 | 1.767 $\pm$ 0.041 | 0.675 $\pm$ 0.031                   | 1.737 $\pm$ 0.034 | 0.677 $\pm$ 0.022 |
|             | Overall      | <b>1.740 <math>\pm</math> 0.015</b> | <b>0.635 <math>\pm</math> 0.014</b> | 1.785 $\pm$ 0.025 | 0.649 $\pm$ 0.016 | 1.842 $\pm$ 0.034 | 0.650 $\pm$ 0.018                   | 1.812 $\pm$ 0.022 | 0.653 $\pm$ 0.015 |
| ETTh2       | 192          | <b>1.726 <math>\pm</math> 0.039</b> | <b>0.202 <math>\pm</math> 0.003</b> | 1.789 $\pm$ 0.014 | 0.206 $\pm$ 0.002 | 1.857 $\pm$ 0.044 | 0.222 $\pm$ 0.007                   | 1.841 $\pm$ 0.009 | 0.217 $\pm$ 0.001 |
|             | Overall      | <b>1.765 <math>\pm</math> 0.030</b> | <b>0.228 <math>\pm</math> 0.002</b> | 1.824 $\pm$ 0.015 | 0.229 $\pm$ 0.002 | 1.880 $\pm$ 0.034 | 0.244 $\pm$ 0.008                   | 1.874 $\pm$ 0.008 | 0.242 $\pm$ 0.004 |
| ETTh1       | 192          | <b>1.759 <math>\pm</math> 0.046</b> | 0.534 $\pm$ 0.022                   | 1.933 $\pm$ 0.069 | 0.536 $\pm$ 0.017 | 1.808 $\pm$ 0.069 | <b>0.504 <math>\pm</math> 0.021</b> | 1.852 $\pm$ 0.046 | 0.537 $\pm$ 0.026 |
|             | Overall      | <b>1.824 <math>\pm</math> 0.020</b> | 0.562 $\pm$ 0.022                   | 2.038 $\pm$ 0.031 | 0.568 $\pm$ 0.014 | 1.878 $\pm$ 0.030 | <b>0.538 <math>\pm</math> 0.016</b> | 1.943 $\pm$ 0.025 | 0.563 $\pm$ 0.013 |
| ETTh2       | 192          | <b>1.817 <math>\pm</math> 0.045</b> | <b>0.261 <math>\pm</math> 0.011</b> | 1.977 $\pm$ 0.036 | 0.289 $\pm$ 0.009 | 1.923 $\pm$ 0.036 | 0.275 $\pm$ 0.014                   | 1.890 $\pm$ 0.050 | 0.284 $\pm$ 0.008 |
|             | Overall      | <b>1.936 <math>\pm</math> 0.026</b> | <b>0.274 <math>\pm</math> 0.007</b> | 2.087 $\pm$ 0.030 | 0.307 $\pm$ 0.005 | 1.994 $\pm$ 0.022 | 0.295 $\pm$ 0.008                   | 2.027 $\pm$ 0.023 | 0.309 $\pm$ 0.006 |
| Weather     | 192          | <b>2.914 <math>\pm</math> 0.081</b> | <b>2.885 <math>\pm</math> 0.650</b> | 2.974 $\pm$ 0.059 | 3.935 $\pm$ 0.334 | 3.025 $\pm$ 0.065 | 5.423 $\pm$ 0.822                   | 2.914 $\pm$ 0.026 | 3.059 $\pm$ 0.622 |
|             | Overall      | <b>3.168 <math>\pm</math> 0.058</b> | <b>2.376 <math>\pm</math> 0.323</b> | 3.265 $\pm$ 0.039 | 3.643 $\pm$ 0.186 | 3.276 $\pm$ 0.037 | 4.624 $\pm$ 0.411                   | 3.245 $\pm$ 0.041 | 2.946 $\pm$ 0.574 |
| Electricity | 192          | <b>6.041 <math>\pm</math> 0.017</b> | <b>0.528 <math>\pm</math> 0.030</b> | 6.074 $\pm$ 0.033 | 0.563 $\pm$ 0.010 | 6.090 $\pm$ 0.022 | 0.627 $\pm$ 0.028                   | 6.049 $\pm$ 0.026 | 0.588 $\pm$ 0.020 |
|             | Overall      | <b>6.040 <math>\pm</math> 0.011</b> | <b>0.526 <math>\pm</math> 0.014</b> | 6.079 $\pm$ 0.018 | 0.564 $\pm$ 0.014 | 6.107 $\pm$ 0.019 | 0.631 $\pm$ 0.018                   | 6.057 $\pm$ 0.015 | 0.579 $\pm$ 0.011 |

two models “Encoder-only TSFM” and “Decoder-only TSFM” in our following experiments.

**Datasets.** Following Yao et al. [35], we train “Encoder-only TSFM” and “Decoder-only TSFM” on the LOTSA dataset in the pretraining stage. These models are then evaluated on the out-of-distribution LSF dataset [32], using various prediction lengths and a preprocessing pipeline as in Yao et al. [35]. We evaluate 6 out-of-distribution datasets in LSF [32] (ETTh1, ETTh2, ETTh1, ETTh2, Weather, Electricity). We also take a very small number of samples (32 in all our experiments) from these evaluation datasets as a validation set used by TSIS (held out from the test set). The training dataset of the high-quality prompt-guided diffusion model is a very small subset sampled from the training dataset of TSFM. In our experiments, we sample 5% of the training data.

**Baselines.** We examine two popular data augmentation methods for TSFM. “TSMixup” or Time Series Mixup is proposed in Ansari et al. [2] which creates new data samples from  $k$  existing ones through weighted summation. The generation process can be represented as  $\hat{z} = \sum_{i=1}^k \lambda_i z_i$ , where  $\hat{z}$  is the generated sample and  $\lambda_i$  is the weight for the  $i^{\text{th}}$  sample. “Jitter” [27] is another widely adopted data augmentation method that involves small noise on existing samples to increase the robustness and diversity. The generation process can be represented as  $\hat{z} = z + \epsilon$ , where  $\epsilon \sim \mathcal{N}(0, \sigma)$  and  $\sigma$  is the noise variance. We also include the result of the regular training process without additional data augmentation.

**Evaluation Metrics.** To be consistent with Yao et al. [35], we primarily report the normalized mean absolute percentage error (MAPE) and negative log-likelihood (NLL) for Encoder-only TSFM and Decoder-only TSFM, as these metrics avoid distortions caused by high-amplitude samples. We specifically report the metrics for prediction length to be 192 and “overall” (average performance of using prediction lengths).

## 4.2 Performance of OATS

In Table 1 and Table 2, we present the NLL and MAPE on the test datasets on Encoder-only TSFM and Decoder-only TSFM, respectively. OATS outperforms baselines as well as regular training in almost all cases and both metrics. Especially, for datasets ETTh1, ETTh2, ETTh2, Weather and Electricity, OATS reaches the best result. TSMixup and Jitter get mixed performance compared to regular training, as presented by the light green and red backgrounds. While OATS performs more consistently better than regular training. In Figure 3, we present the test loss (NLL) curve on evaluation datasets on Encoder-only TSFM and different test datasets. OATS achieves a faster reduction in NLL than other baseline data augmentation methods, as well as the regular training, and achieves better overall performance. Additional results are provided in Appendix D.

## 4.3 Ablation Studies

In this section, we perform a variety of ablation studies to present the effect of several core components: validation set  $\mathcal{D}_{val}$  (Section 4.3.1, TSIS guidance (Section 4.3.2), and generation strategy (Section 4.3.3).

**4.3.1 Effect of Validation Set.** We evaluate OATS under two practical scenarios: a few-shot setting where the validation set is drawn from the target domain, and an in-domain setting where it is drawn from held-out training data. Table 3 shows that both settings outperform regular training. The few-shot setting provides stronger guidance as expected.

**4.3.2 Effect of TSIS Guidance.** To isolate the contribution of TSIS, we compare OATS with a variant that uses the same conditional diffusion model but with randomly selected prompts instead of TSIS-selected ones. As shown in Table 4, TSIS-guided generation outperforms random-guided generation in NLL across all three datasets, and in MAPE on ETTh1 and ETTh1. On Electricity MAPE,

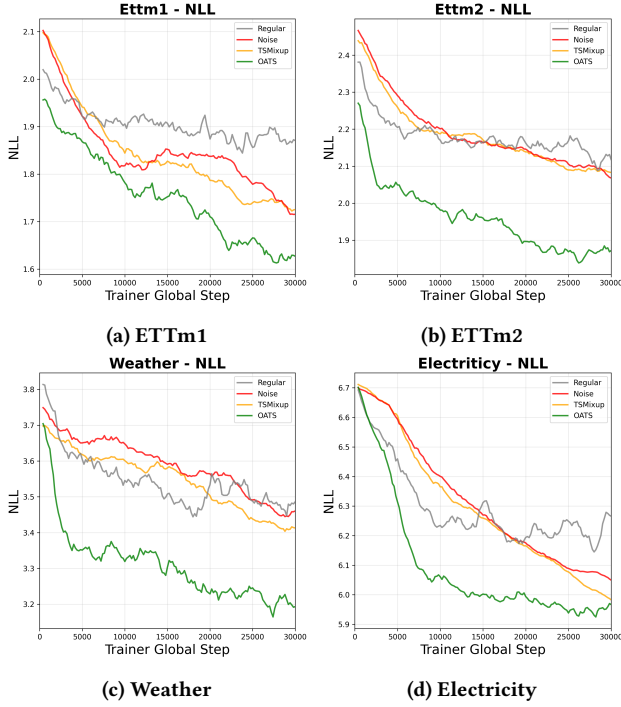


Figure 3: Test loss (NLL) of OATS, TSMixup, Jitter and Regular training for each training step.

Table 3: Effect of validation set source. “Few-shot” uses a small number of samples from the target domain as the validation set. “In-domain” uses held-out samples from the training data as the validation set. Cells are colored **green** if better than Regular and **red** if worse.

| Dataset | Pred. length | OATS (few-shot) |       | OATS (in-domain) |       | Dataset     | Pred. length | OATS (few-shot) |       | OATS (in-domain) |       |
|---------|--------------|-----------------|-------|------------------|-------|-------------|--------------|-----------------|-------|------------------|-------|
|         |              | NLL             | MAPE  | NLL              | MAPE  |             |              | NLL             | MAPE  | NLL              | MAPE  |
| ETTh1   | 192          | 1.627           | 0.672 | 1.781            | 0.761 | ETTh2       | 192          | 1.857           | 0.267 | 1.915            | 0.322 |
|         | Overall      | 1.614           | 0.623 | 1.728            | 0.670 |             | Overall      | 1.876           | 0.263 | 1.910            | 0.312 |
| ETTh2   | 192          | 1.872           | 0.208 | 1.830            | 0.206 | Weather     | 192          | 3.193           | 1.778 | 3.304            | 1.821 |
|         | Overall      | 1.863           | 0.222 | 1.838            | 0.221 |             | Overall      | 3.216           | 1.659 | 3.319            | 1.545 |
| ETTh1   | 192          | 1.794           | 0.681 | 1.827            | 0.720 | Electricity | 192          | 5.967           | 0.515 | 6.032            | 0.545 |
|         | Overall      | 1.814           | 0.709 | 1.840            | 0.777 |             | Overall      | 5.945           | 0.507 | 6.013            | 0.604 |

the two are comparable, suggesting that the benefit of TSIS guidance is more pronounced on the likelihood metric.

**4.3.3 Effect of Generation Strategy.** We compare a number of augmentation strategies to understand how each design choice contributes to OATS’s performance. Table 4 shows a clear progression: selection only improves over regular training but remains limited by data diversity; unconditional improves by introducing synthetic variety while suffering from low-quality samples; and TSIS-guided conditional generation achieves the best results by combining targeted selection with high-fidelity generation. In addition, we also included AutoTCL [38] as a generation strategy baseline.

Table 4: Ablation study on the effect of generation strategy and TSIS guidance. “Selection only” trains on TSIS-selected samples (with duplication if needed to fill the batch) without generation. “Unconditional” uses a single unconditional diffusion model. “Random-guided” uses the same conditional diffusion model but with randomly selected prompts instead of TSIS-selected ones. AutoTCL [38] is a baseline generation method. OATS uses the conditional diffusion model with TSIS-selected prompts. Cells are colored **green** if better than Regular and **red** if worse.

| Dataset     | Pred. length | OATS  |       | Random-guided |       | Selection only |       | Unconditional |       | AutoTCL |       |
|-------------|--------------|-------|-------|---------------|-------|----------------|-------|---------------|-------|---------|-------|
|             |              | NLL   | MAPE  | NLL           | MAPE  | NLL            | MAPE  | NLL           | MAPE  | NLL     | MAPE  |
| ETTh1       | 192          | 1.794 | 0.681 | 1.831         | 0.767 | 1.874          | 0.793 | 1.811         | 0.617 | 1.811   | 0.933 |
|             | Overall      | 1.814 | 0.709 | 1.853         | 0.824 | 1.899          | 0.836 | 1.827         | 0.632 | 1.825   | 1.024 |
| ETTh2       | 192          | 1.627 | 0.672 | 1.766         | 0.859 | 1.804          | 0.815 | 1.782         | 0.811 | 1.821   | 0.844 |
|             | Overall      | 1.614 | 0.623 | 1.729         | 0.778 | 1.742          | 0.703 | 1.723         | 0.732 | 1.753   | 0.739 |
| Electricity | 192          | 5.967 | 0.515 | 6.221         | 0.470 | 6.147          | 0.632 | 6.151         | 0.538 | 6.207   | 0.513 |
|             | Overall      | 5.945 | 0.507 | 6.174         | 0.509 | 6.156          | 0.648 | 6.114         | 0.576 | 6.114   | 0.501 |

#### 4.4 Case studies

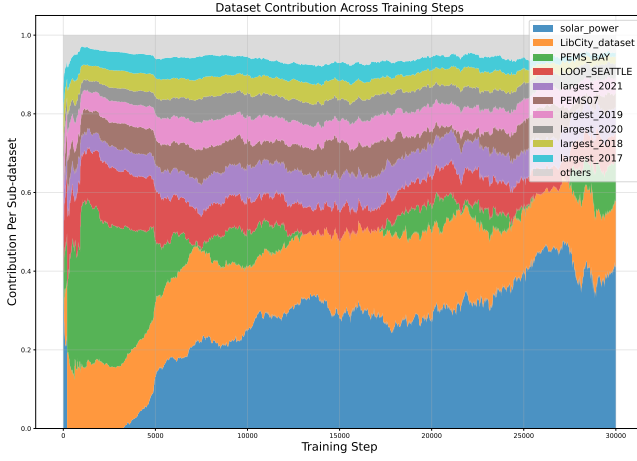
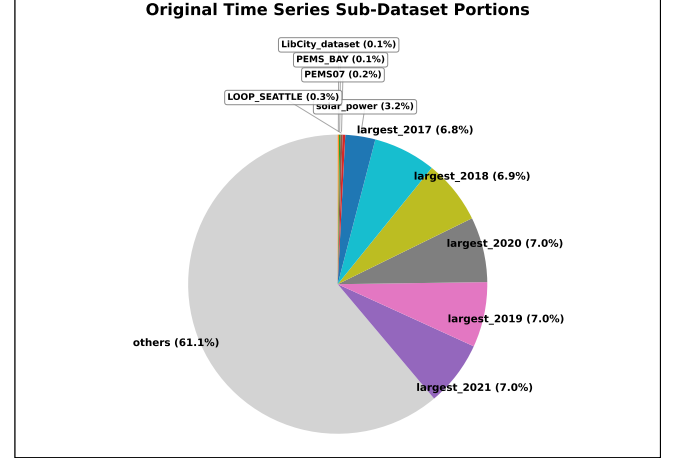
In this subsection, we carry out two case studies to further provide intuitive evidence of our motivation and some insights.

**Contribution of each sub-dataset.** In the first case study, we present the per-sub-dataset TSIS, i.e.,  $\Phi_{D_i}$ , throughout the training process (Figure 4a), along with the original proportion of each sub-dataset in the training corpus (Figure 4b). The results reveal two key insights: (1) the contribution of each sub-dataset evolves during training, which intuitively motivates the use of online data augmentation; and (2) the contribution of a training sub-dataset does not necessarily align with its size. For instance, a relatively small sub-dataset (e.g., solar\_power) can contribute substantially to reducing test loss, whereas some large sub-datasets (e.g., those grouped under others, which account for 60% of the total data in Figure 4b) provide only a limited contribution (around 10% in Figure 4a). Such findings are difficult to uncover through heuristic design or empirical intuition alone, but could be revealed through principled influence analysis.

**Selected high-quality prompts and generated samples.** In the second case study, we present the high-quality data samples prompts selected according to TSIS, and the corresponding guided generation in Figure 5. The results show that the generation model of the high-quality guided data augmentation could generate realistic data samples similar to the prompt with various patterns.

#### 5 Conclusion

In this paper, we introduce OATS, a principled strategy for generating synthetic data tailored to different training steps in TSFMs. The core idea behind OATS is to leverage valuable training samples as principled guiding signals and dynamically generate high-quality synthetic data conditioned on them. Empirical evaluations across diverse model architectures, datasets, and prediction lengths demonstrate that OATS substantially outperforms both standard data augmentation methods and regular training in terms of TSFM

(a) Sub-dataset TSIS ( $\Phi_{D_i}$ ) along the training.

(b) Sub-dataset portion in training dataset.

Figure 4: Comparison between sub-dataset data contribution and data portion.

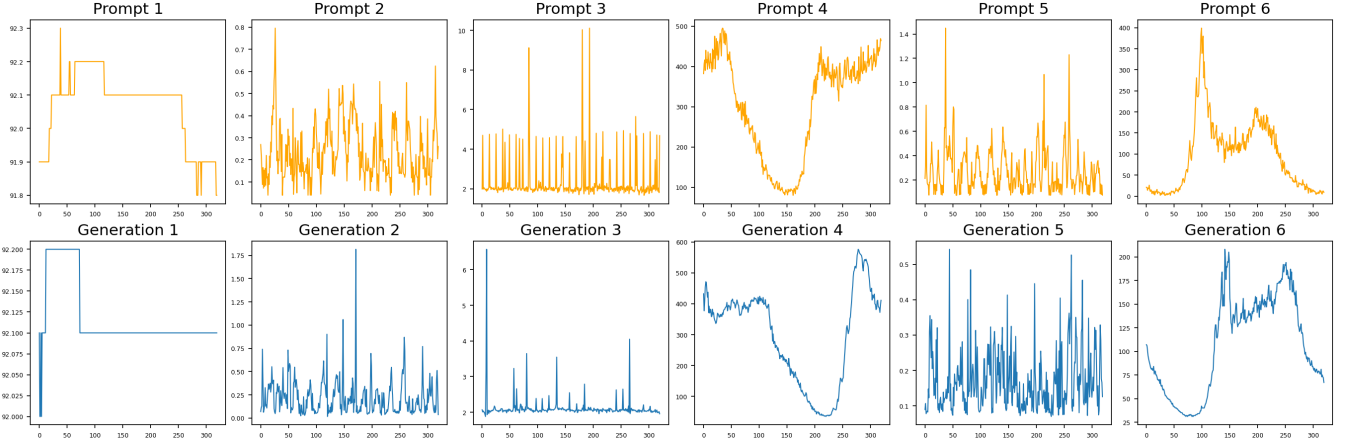


Figure 5: (First row) High-quality data samples prompt. (Second row) The corresponding guided generation.

performance. By expanding the scope of online TSFM data augmentation, OATS enables researchers to systematically optimize TSFM training datasets in a principled way.

**Limitation.** OATS introduces additional computational overhead from TSIS gradient evaluation and diffusion-based sampling compared to regular training, though this can be mitigated in practice (see Appendix A). The method also depends on a small validation set to define data quality, and its current evaluation is limited to forecasting; generalization to other time series tasks such as classification and anomaly detection remains future work.

## References

- [1] Abdul Fatir Ansari, Oleksandr Shchur, Jaris Kücken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Guerron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, Huzefa Rangwala, George Karypis, Yuyang Wang, and Michael Bohlke-Schneider. 2025. Chronos-2: From Univariate to Universal Forecasting. *arXiv:2510.15821 [cs.LG]* <https://arxiv.org/abs/2510.15821>
- [2] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. 2024. Chronos: Learning the Language of Time Series. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=gerNCVqqtR>
- [3] Junwei Deng, Yuzheng Hu, Pingbang Hu, Ting-Wei Li, Shixuan Liu, Jiachen T. Wang, Dan Ley, Qirun Dai, Benhao Huang, Jin Huang, Cathy Jiao, Hoang Anh Just, Yijun Pan, Jingyan Shen, Yiwen Tu, Weiye Wang, Xinhe Wang, Shichang Zhang, Shiyuan Zhang, Ruoxi Jia, Himabindu Lakkaraju, Hao Peng, Weijing Tang, Chenyan Xiong, Jieyu Zhao, Hanghang Tong, Han Zhao, and Jiaqi W. Ma. 2025. A Survey of Data Attribution: Methods, Applications, and Evaluation in the Era of Generative AI. (2025). doi:10.2139/ssrn.5451054 Available at SSRN: <https://ssrn.com/abstract=5451054> or <http://dx.doi.org/10.2139/ssrn.5451054>.
- [4] Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. 2021. Timevae: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095* (2021).
- [5] Samuel Dooley, Gurnoor Singh Khurana, Chirag Mohapatra, Siddhartha Naidu, and Colin White. 2023. ForecastPFN: synthetically-trained zero-shot forecasting. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '23). Curran Associates Inc., Red Hook, NY, USA, Article 112, 24 pages.

- [6] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027* (2020).
- [7] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. 2024. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885* (2024).
- [8] Lin Lawrence Guo, Ethan Steinberg, Scott Lanyon Fleming, Jose Posada, Joshua Lemmon, Stephen R Pfohl, Nigam Shah, Jason Fries, and Lillian Sung. 2023. EHR foundation models improve robustness in the presence of temporal distribution shift. *Scientific Reports* 13, 1 (2023), 3767.
- [9] Yu-Hao Huang, Chang Xu, Yueying Wu, Wu-Jun Li, and Jiang Bian. 2025. Timedp: Learning to generate multi-domain time series with domain prompts. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 17520–17527.
- [10] Heikki Junninen, Harri Niska, Kari Tuppurainen, Juhani Ruuskanen, and Mikko Kolehmainen. 2004. Methods for imputation of missing values in air quality data sets. *Atmospheric environment* 38, 18 (2004), 2895–2907.
- [11] Raehyun Kim, Chan Ho So, Minbyul Jeong, Sanghoon Lee, Jinkyu Kim, and Jaewoo Kang. 2019. Hats: A hierarchical graph attention network for stock movement prediction. *arXiv preprint arXiv:1908.07999* (2019).
- [12] Pang Wei Koh and Percy Liang. 2017. Understanding black-box predictions via influence functions. In *International conference on machine learning*. PMLR, 1885–1894.
- [13] Kseniia Kuvshinova, Olga Tsymboi, Alina Kostromina, Dmitry Simakov, and Elizaveta Kovtun. 2024. Towards Foundation Time Series Model: To Synthesize Or Not To Synthesize? *arXiv preprint arXiv:2403.02534* (2024).
- [14] Tian Lan, Hao Duong Le, Jinbo Li, Wenjun He, Meng Wang, Chenghao Liu, and Chen Zhang. 2025. Towards Foundation Models for Zero-Shot Time Series Anomaly Detection: Leveraging Synthetic Data and Relative Context Discrepancy. *arXiv:2509.21190* [cs.LG] <https://arxiv.org/abs/2509.21190>
- [15] Junjie Li, Yang Liu, Weiqing Liu, Shikai Fang, Lewen Wang, Chang Xu, and Jiang Bian. 2024. MarS: a Financial Market Simulation Engine Powered by Generative Foundation Model. *arXiv preprint arXiv:2409.07486* (2024).
- [16] Yuxuan Liang, Yutong Xia, Songyu Ke, Yiwei Wang, Qingsong Wen, Junbo Zhang, Yu Zheng, and Roger Zimmermann. 2023. Airformer: Predicting nationwide air quality in china with transformers. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 14329–14337.
- [17] Shanchuan Lin, Bingchen Liu, Jiashi Li, and Xiao Yang. 2023. Common diffusion noise schedules and sample steps are flawed. 2024 IEEE. In *CVF Winter Conference on Applications of Computer Vision (WACV)*. 5392–5399.
- [18] Xu Liu, Taha Aksu, Juncheng Liu, Qingsong Wen, Yuxuan Liang, Caiming Xiong, Silvio Savarese, Doyen Sahoo, Junnan Li, and Chenghao Liu. 2025. Empowering Time Series Analysis with Synthetic Data: A Survey and Outlook in the Era of Foundation Models. *arXiv preprint arXiv:2503.11411* (2025).
- [19] Vladyslav Moroshan, Julien Siems, Arber Zela, Timur Carstensen, and Frank Hutter. 2025. TempoPFN: Synthetic Pre-training of Linear RNNs for Zero-shot Time Series Forecasting. *arXiv:2510.25502* [cs.LG] <https://arxiv.org/abs/2510.25502>
- [20] Sai Shankar Narasimhan, Shubhankar Agarwal, Oguzhan Akcin, Sujay Sanghavi, and Sandeep Chinchali. 2024. Time weaver: A conditional time series generation model. *arXiv preprint arXiv:2403.02682* (2024).
- [21] Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. 2020. Estimating training data influence by tracing gradient descent. *Advances in Neural Information Processing Systems* 33 (2020), 19920–19930.
- [22] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [23] Sebastian Ruder. 2016. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747* (2016).
- [24] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. 2024. Time-moe: Billion-scale time series foundation models with mixture of experts. *arXiv preprint arXiv:2409.16040* (2024).
- [25] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising Diffusion Implicit Models. *CoRR abs/2010.02502* (2020). *arXiv:2010.02502* <https://arxiv.org/abs/2010.02502>
- [26] Ege Onur Taga, Halil Alperen Gozeten, Kutay Tire, Rahul Dalvi, Reinhard Heckel, and Samet Oymak. 2025. Filter, Augment, Forecast: Online Data Selection for Robust Time Series Forecasting. In *1st ICML Workshop on Foundation Models for Structured Data*. <https://openreview.net/forum?id=YYsXgr8MUQ>
- [27] Terry T Um, Franz M Pfister, Daniel Pichler, Satoshi Endo, Muriel Lang, Sandra Hirche, Urban Fietzek, and Dana Kulić. 2017. Data augmentation of wearable sensor data for parkinson's disease monitoring using convolutional neural networks. In *Proceedings of the 19th ACM international conference on multimodal interaction*. 216–220.
- [28] Jiachen T Wang, Prateek Mittal, Dawn Song, and Ruoxi Jia. 2024. Data shapley in one training run. *arXiv preprint arXiv:2406.11011* (2024).
- [29] Jiachen T Wang, Dawn Song, James Zou, Prateek Mittal, and Ruoxi Jia. 2024. Capturing the temporal dependence of training data influence. *arXiv preprint arXiv:2412.09538* (2024).
- [30] Jiachen Tianhao Wang, Tong Wu, Dawn Song, Prateek Mittal, and Ruoxi Jia. 2024. Greats: Online selection of high-quality data for llm training in every iteration. *Advances in Neural Information Processing Systems* 37 (2024), 131197–131223.
- [31] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified training of universal time series forecasting transformers. (2024).
- [32] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *The Eleventh International Conference on Learning Representations*. [https://openreview.net/forum?id=ju\\_Uqw384Oq](https://openreview.net/forum?id=ju_Uqw384Oq)
- [33] Shifeng Xie, Vasili Feofanov, Marius Alonso, Ambroise Odonnat, Jianfeng Zhang, Themis Palpanas, and Ievgen Redko. 2025. CauKer: classification time series foundation models can be pretrained on synthetic data only. *arXiv:2508.02879* [cs.LG] <https://arxiv.org/abs/2508.02879>
- [34] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy S Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. 2023. Doremi: Optimizing data mixtures speeds up language model pretraining. *Advances in Neural Information Processing Systems* 36 (2023), 69798–69818.
- [35] Qingren Yao, Chao-Han Huck Yang, Renhe Jiang, Yuxuan Liang, Ming Jin, and Shirui Pan. 2024. Towards Neural Scaling Laws for Time Series Foundation Models. *arXiv preprint arXiv:2410.12360* (2024).
- [36] Jinsung Yoon, Daniel Jarrett, and Mihaela Van der Schaar. 2019. Time-series generative adversarial networks. *Advances in neural information processing systems* 32 (2019).
- [37] Zahra Zamanzadeh Darban, Geoffrey I Webb, Shirui Pan, Charu Aggarwal, and Mahsa Salehi. 2024. Deep learning for time series anomaly detection: A survey. *Comput. Surveys* 57, 1 (2024), 1–42.
- [38] Xu Zheng, Tianchun Wang, Wei Cheng, Aitian Ma, Haifeng Chen, Mo Sha, and Dongsheng Luo. 2024. Parametric Augmentation for Time Series Contrastive Learning. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=EIPLdFy3vp>

## A Explore & Exploit Mechanism

Ideally,  $\mathcal{H}_t$  should be identified from the whole training dataset  $\mathcal{D}_{tr}$  in Section 3.2, while the process of calculating TSIS for all training samples will be time-consuming. We are inspired by the multi-armed bandit problem and design an explore-exploit mechanism that could reuse the TSIS calculated for samples of previous training steps as well as explore new samples.

The mechanism divides training steps into *explore* steps and *exploit* steps. In *explore* steps, we calculate TSIS for a batch of data samples  $\mathcal{B}_t$  sampled by strategy  $\pi_r$ . We assume the locality of TSIS among  $z$  in the same subset. Based on this assumption, we maintain a dynamic cache  $\Phi_{\mathcal{D}_l} \in \mathbb{R}, l \in [L]$  for each subset and update it through exponentially moving average. The partition of  $\mathcal{D}_{tr} = \bigcup_{l=1}^L \mathcal{D}_l$  can naturally be the sub-datasets of a large collection or some clustering results. The update of  $\Phi_{\mathcal{D}_l}$  on training step  $t$  can be represented as

$$\Phi_{\mathcal{D}_l} = (1 - \beta)\Phi_{\mathcal{D}_l} + \beta \sum_{z_i \in \mathcal{D}_l \cap \mathcal{B}_t} \mathcal{F}_{\mathcal{D}_{val}}(z_i, w_t) / |\mathcal{D}_l \cap \mathcal{B}_t| \quad (4)$$

where  $\mathcal{B}_t$  is a sample of size  $|\mathcal{B}_t|$  from  $\pi_r$ , and  $\pi_r$  is a uniform sampling strategy  $\mathcal{U}(\mathcal{D}_{tr})$ ,  $\beta \in [0, 1]$  is the hyperparameter controls the decay factor of the exponentially moving average. The design of  $\pi_r$  for exploring steps reflects the spirit to visit new data points and update the TS data. It randomly samples from the full training dataset  $\mathcal{D}_{tr}$ . Additionally, we refer to the influence score part of the TSIS in this subsection when we use  $\mathcal{F}_{\mathcal{D}_{val}}$ .

For the *exploit* step, we design a sample algorithm  $\pi_o$  utilizing the  $\Phi_{\mathcal{D}_l}$  cached in *explore* step. The design of  $\pi_o$  reflects the latest estimation of each subset's quality.

$$\pi_o = \mathcal{U}(\mathcal{D}_l) \text{ with probability } \frac{|\mathcal{D}_l| \cdot \max(0, \Phi_{\mathcal{D}_l})}{\sum_k |\mathcal{D}_k| \cdot \max(0, \Phi_{\mathcal{D}_k})} \quad (5)$$

The choice of *explore* or *exploit* step is handled by  $\epsilon$ -greedy, a popular strategy for explore and exploit. We design a strategy  $\psi$  that is defined to be “explore” with probability  $\epsilon$  and “exploit” with probability  $(1 - \epsilon)$ , where  $\epsilon$  is a hyperparameter controlling the balance between explore and exploit. Notably, the overhead of the *exploit* step is small enough to be ignored since TSIS is not calculated in such steps. A more careful complexity analysis is introduced in Appendix B.

### A.1 Algorithm with Explore-exploit

We summarize Online Data Augmentation for Time Series Foundation Models (OATS) and present in Algorithm 2.

### A.2 Performance of OATS under Different Explore-Exploit Levels

The key experiment goal of this subsection is to examine the sensitivity and behavior of OATS on different levels of explore-exploit mechanisms through adjusting the value of  $\epsilon$ .  $\epsilon$  is a hyperparameter designed to control the possibility of carrying out *explore* ( $\epsilon$ ) or *exploit* step ( $1 - \epsilon$ ). *Explore* step will examine new data by calculating the TSIS ( $\mathcal{F}_{\mathcal{D}_{val}}$ ), which is time-consuming. *Exploit* step will leverage cached TSIS ( $\Phi_{\mathcal{D}_l}$ ) calculated in previous steps to directly sample a high-quality batch, which is light-weighted.

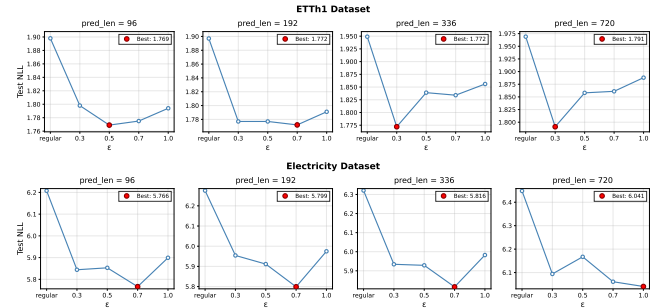
### Algorithm 2 Online Data Augmentation for Time Series Foundation Models (OATS)

**Require:** Training dataset  $\mathcal{D}_{tr} = \bigcup_{l=1}^L \mathcal{D}_l$ , validation dataset  $\mathcal{D}_{val}$ , batch size  $b$ , augmentation algorithm  $\mathcal{G}$ , loss  $\ell$ , strategy  $\psi$ , sampling algorithm  $\pi_r/\pi_o$ , TSIS function  $\mathcal{F}_{\mathcal{D}_{val}}$ .

- 1: Initialize model  $w_0$
- 2: Initialize  $\Phi_{\mathcal{D}_l}, l \in [L]$  to be subset proportion.
- 3: **for**  $t = 1$  to  $T$  **do**
- 4:   **if**  $\psi(t) == \text{“explore”}$  **then**
- 5:     Sample  $\mathcal{B}_t \sim \pi_r^b$  // Step 1
- 6:     Calculate  $\mathcal{F}_{\mathcal{D}_{val}}(z_i, w_t)$  for  $z_i \in \mathcal{B}_t$
- 7:     Update  $\Phi_{\mathcal{D}_l}$  according to Eq. 4.
- 8:     Select subset  $\mathcal{H}_t \subseteq \mathcal{B}_t$  with top- $\lfloor b/2 \rfloor$  of  $\mathcal{F}_{\mathcal{D}_{val}}$ .
- 9:     Generate  $\mathcal{S}_t$  of size  $\lfloor b/2 \rfloor$  via  $\mathcal{G}$  guided by  $\mathcal{H}_t$ . // Step 2
- 10:   **end if**
- 11:   **if**  $\psi(t) == \text{“exploit”}$  **then**
- 12:     Sample  $\mathcal{H}_t \sim \pi_o^{\lfloor b/2 \rfloor}$  // Step 1
- 13:     Generate  $\mathcal{S}_t$  of size  $\lfloor b/2 \rfloor$  via  $\mathcal{G}$  guided by  $\mathcal{H}_t$ . // Step 2
- 14:   **end if**
- 15:   Update  $w_t$  on mini-batch  $\mathcal{H}_t \cup \mathcal{S}_t$  to get  $w_{t+1}$ .
- 16: **end for**

In Figure 6 we present the performance of OATS on different levels of explore-exploit mechanisms. We choose to set  $\epsilon = [0.3, 0.5, 0.7, 1.0]$  and report the performance on two test datasets (ETTh1 and Electricity) and four prediction lengths (96, 192, 336, 720).

The best performance in each setting is often not achieved by  $\epsilon = 1$  (all explore step), the most computationally heavy setting that calculates TSIS in every step. This shows that leveraging cached TSIS is helpful to select higher-quality batch (using  $\pi_o$ ) to guide the data augmentation and demonstrates the potential of using the mechanism to reduce computational overhead. Another observation also shows that OATS consistently performs better than the regular training process regardless of the setting of  $\epsilon$ . We included a detailed complexity and time cost analysis in Appendix B.



**Figure 6: Performance of OATS on different explore-exploit ratio  $\epsilon$ . (First Row) Test NLL on ETTh1. (Second Row) Test NLL on Electricity. The best performance is labeled by a red circle.**

## B Complexity Analysis

OATS involves computational overhead to the training process, which can be spitted into two sources: 1) calculation of TSIS and 2) synthetic data generation. **In summary**, the overhead of TSIS



calculation is up to  $\epsilon \times$  of the regular training step. The good performance shown in Figure 6 with  $\epsilon = 0.3$  shows that this overhead could be relatively small. The overhead of synthetic data generation model is a fixed number, which gets lower as the regular training step has higher time cost, e.g., a larger TSFM with more parameters.

**Calculation of TSIS.** For TSIS calculation, the overhead of Equation 1 mainly comes from the gradient calculation of validation data samples in “explore” steps. In our experiment, we use a small number of validation data samples (32) that matches the training batch size. This helps maintain the same computation complexity between TSIS and the regular gradient descent training step as  $O(b)$ , where  $b$  is the training batch size. Additionally, we also adopted an efficient implementation called “Ghost Inner Product” [28] to avoid per-sample gradient calculation while get the dot product result.

**LEMMA 2 (GHOST INNER PRODUCT).** *In the influence function calculation in TSIS, which includes sample-level gradient dot products, i.e.,  $\nabla_{\mathbf{w}} \ell(\mathbf{z}^{(1)}, \mathbf{w}_t) \cdot \nabla_{\mathbf{w}} \ell(\mathbf{z}^{(2)}, \mathbf{w}_t)$ . We may only perform the back-propagation once and calculate the gradient dot product. Ghost Inner Product perform the product layer by layer in a model and here we present the method for a simple linear layer.*

$$\begin{aligned} \frac{\partial \ell^{(1)}}{\partial \mathbf{w}} \cdot \frac{\partial \ell^{(2)}}{\partial \mathbf{w}} &= \left( \mathbf{a}^{(1)} \otimes \frac{\partial \ell^{(1)}}{\partial \mathbf{s}^{(1)}} \right) \cdot \left( \mathbf{a}^{(2)} \otimes \frac{\partial \ell^{(2)}}{\partial \mathbf{s}^{(2)}} \right) \\ &= \left( (\mathbf{a}^{(1)})^\top \mathbf{a}^{(2)} \right) \left( \left( \frac{\partial \ell^{(1)}}{\partial \mathbf{s}^{(1)}} \right)^\top \frac{\partial \ell^{(2)}}{\partial \mathbf{s}^{(2)}} \right). \end{aligned} \quad (6)$$

where  $\mathbf{a}^{(1)}$  and  $\mathbf{a}^{(2)}$  is the linear layer’s input,  $\mathbf{s}^{(1)}$  and  $\mathbf{s}^{(2)}$  is the pre-activation output, and  $\otimes$  indicates the outer product.

In TSIS,  $\ell^{(1)} = \ell(\mathbf{z}_i, \mathbf{w}_t)$  in each training step  $t$  and  $\mathbf{z}_i \in \mathcal{B}_t$  and  $\ell^{(2)} = \ell(\mathcal{D}_{val}, \mathbf{w}_t)$ . “Ghost Inner Product” enable the implementation of influence score in TSIS to carry out **only one additional backpropagation** on  $\ell(\mathcal{B}_t, \mathbf{w}_t) + \ell(\mathcal{D}_{val}, \mathbf{w}_t)$  to get all terms in Equation 6 compared to regular training.

Furthermore, the overhead of “exploit” steps is small enough to be ignored since TSIS in Equation 1 is not used in those steps, **which means that the computational overhead of TSIS will be reduced by  $(1-\epsilon)$  and memory footprint will be on par with the regular training.** In Figure 6,  $\epsilon = 0.3$  also substantially outperform regular training process, which shows good potential of reducing overhead. Following table 5 is a time cost record for training batch size 32. It is worth noting that the actual implementation may affect the time cost.

**Table 5: Per-training step time cost under different settings of Explore-exploit ratio  $\epsilon$ .**

| $\epsilon$                             | Regular | 0.3  | 0.5  | 0.7  | 1.0  |
|----------------------------------------|---------|------|------|------|------|
| Avg. time cost per step (s) on A40 GPU | 0.74    | 0.95 | 1.09 | 1.20 | 1.32 |

**Synthetic data generation.** For synthetic data generation through diffusion model sampling, it is hard to compare the complexity

with the regular gradient descent training step. Empirically, we find that some accelerated sampling strategies like DDIM [25] could perform well enough with very small sample steps. Moreover, the generation overhead will be a fixed term and the relative proportion will get lower as the TSFMs get larger, e.g., with more parameters. The overhead runtime of current synthetic data generation is 0.022s per synthetic sample, which is  $0.022 \times 32 \times 0.5 = 0.35$ s additional to the Table 5.

**Empirical results at same training time.** We also show the performance of OATS and baselines’ performance under the **same training time as regular training**. OATS still outperform other baselines. It is also worth noting that with a limited training time, OATS may be more far away from converging.

**Table 6: Performance (NLL) of OATS and baselines’ performance under the same time cost.**

| Dataset     | OATS( $\epsilon=0.3$ ) | OATS( $\epsilon=0.5$ ) | OATS( $\epsilon=0.7$ ) | OATS( $\epsilon=1$ ) | TSMixup | Jitter | Regular |
|-------------|------------------------|------------------------|------------------------|----------------------|---------|--------|---------|
| Electricity | <b>6.00</b>            | 6.11                   | 6.02                   | 6.04                 | 6.15    | 6.14   | 6.26    |
| ETTh1       | 1.82                   | <b>1.81</b>            | 1.85                   | 1.83                 | 1.88    | 1.94   | 1.89    |

## C Experiment Settings

**TSFM Models.** We conduct the experiment on the TSFM models with the same configuration as the encoder-only model with a 10M parameter size in Yao et al. [35]. The model is a modified version of Moirai [31], introduced by Yao et al. [35], which incorporates patch embedding, rotary positional embedding, and a mixture of distributions to better adapt to time series forecasting while preserving extensibility.

**TSFM Training Process.** We adopt a similar training setting as in Yao et al. [35] for the experiment. We utilize the AdamW optimizer with a batch size of 32 and a maximum learning rate of  $10^{-3}$  with a linear warm-up of  $10^4$  training steps, followed by cosine decay for the remaining  $2 \times 10^4$  steps.

**Datasets for TSFM.** We pretrain the TSFM on the modified (balanced domain sample, quality filtering) LOTSA-100M dataset in the pretraining stage provided by Yao et al. [35]. We take the native sub-dataset division as subsets in our experiment. These models are then evaluated on the out-of-distribution LSF dataset [32], using various prediction lengths (96, 192, 336) and the same preprocessing pipeline as in Yao et al. [35]. The detailed information of evaluation datasets is stated in Table 7. The datasets are under Apache license 2.0 license.

**Generation Model.** We leverage the architecture in Huang et al. [9] as the backbone model. The denoising network of the diffusion model employed adopts a U-Net architecture comprising 4 up/down sampling blocks with 8 attention heads and the dimension of each head is 64. An additional class embedding with 64 dimension is added to the output of each block.

**Generation Model Training Process.** We train a Latent Diffusion Model for time series generation using 200 diffusion time steps with a linear noise schedule ranging from 0.0005 to 0.1. The model employs L1 loss as the training objective and a dropout probability



**Table 7: Evaluation datasets and properties.**

| Dataset     | Domain  | Frequency | # Prediction Length |
|-------------|---------|-----------|---------------------|
| ETTh1       | Energy  | H         | 96/192/336/720      |
| ETTh2       | Energy  | H         | 96/192/336/720      |
| ETTm1       | Energy  | 15min     | 96/192/336/720      |
| ETTm2       | Energy  | 15min     | 96/192/336/720      |
| Electricity | Energy  | H         | 96/192/336/720      |
| Weather     | Climate | H         | 96/192/336/720      |

of 0.5 to enable classifier-free guidance. We set the base learning rate to 0.001.

*Datasets for Generation Model.* We train the generation model for high-quality guided data augmentation described in Section 3 by a sampled dataset from the training dataset of TSFM. We sample 5% of the dataset in 20 selected subsets (Table 8) in LOTSA-100M as the training set of the diffusion model. The generation length of the diffusion model is set to 320, and the diffusion step is set to 200. We use DDIM for the sampling process with 20 steps.

**Table 8: Training datasets of generation model.**

| Dataset                | Domain    | Frequency |
|------------------------|-----------|-----------|
| CMIP6                  | Climate   | 6H        |
| ERA5                   | Climate   | H         |
| CloudOpsTSF            | CloudOTS  | 5T        |
| Azure VM Traces 2017   | CloudOTS  | 5T        |
| Loop Seattle           | Transport | 5T        |
| PEMS07                 | Transport | 5T        |
| PEMS Bay               | Transport | 5T        |
| Q-Traffic              | Transport | 15T       |
| Largest 2017           | Transport | 5T        |
| Largest 2018           | Transport | 5T        |
| Largest 2019           | Transport | 5T        |
| Largest 2020           | Transport | 5T        |
| Largest 2021           | Transport | 5T        |
| Australian Electricity | Energy    | 30T       |
| Buildings900K          | Energy    | H         |
| Solar Power            | Energy    | 4S        |
| Favorita Sales         | Sales     | D         |
| Wiki-Rolling           | Web       | D         |
| LibCity                | Transport | 5T        |
| OthersLOTSA            | Energy    | H         |

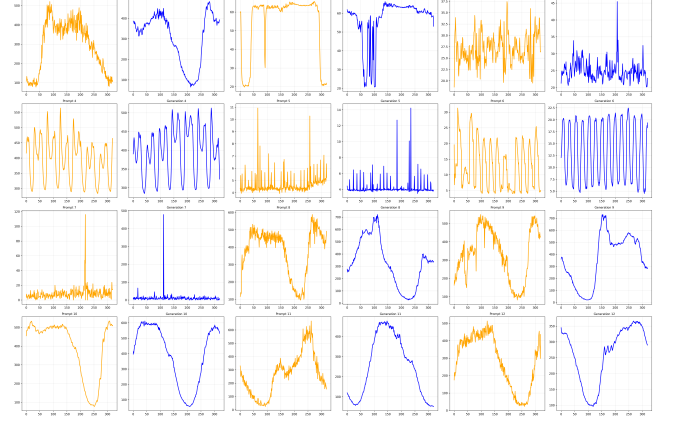
*Hyperparameters.* For experiments in Table 1 and Table 2, we set hyperparameters of TSMixup baseline to have  $k = 2$  and  $\lambda_0 \sim$

$\mathcal{U}(0.1, 0.9)$ ,  $\lambda_1 = 1 - \lambda_0$ . For Jitter, we use  $\sigma = 0.03$ . For OATS, we fixed  $\epsilon = 1$  in Table 1 and Table 2. Experiments with  $\epsilon < 1$  will be included in Figure 6.

*Computation resources.* We run the experiment on A100 GPU and the full GPU hours is around 900 hours.

## D Additional Results

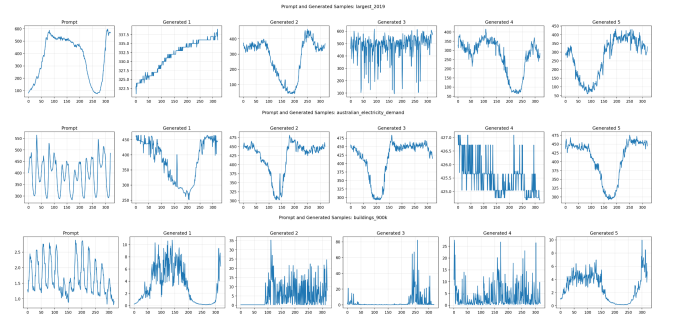
In Figure 7, we present more examples between high-quality data sample prompts and the corresponding generated samples.



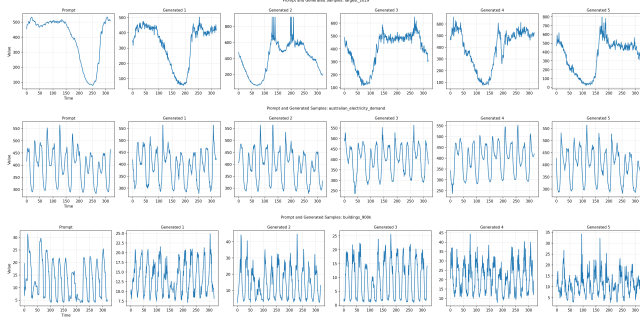
**Figure 7: More examples of high-quality data samples prompts (yellow) and the corresponding guided generation (blue).**

### D.1 Improvement of class embedding on conditional generation.

Figures 8 and 9 demonstrate the improvement brought by class embeddings. In Figure 8, the samples generated without class embeddings (the following five) exhibit broken patterns that fail to align with the prompts (first in each row). In contrast, Figure 9 illustrates that class embeddings help the model achieve better structural alignment with the prompts.



**Figure 8: Examples of data generation of diffusion model without class embedding. For each row, the first sample is prompt and the following five are guided generations.**



**Figure 9: Examples of data generation of diffusion model with class embedding. For each row, the first sample is prompt, and the following five are guided generations.**

## D.2 Sensitivity of initial partition

In Table 9, we follow the native partition structure (sub-datasets) provided in the LOTSA dataset and evaluate different settings of initial data granularity. “Native Subdataset” refers to the default setting used in the paper, where we maintain the original partition structure. “Combine each two” represents a coarser granularity setting where we merge two native subdatasets into a single partition. Additionally, we include a baseline with random partitioning, where data samples are randomly assigned to partitions while maintaining the same total number of partitions as the native subdataset configuration.

**Table 9: Performance comparison of different granularity partitions.**

| Dataset | Pred. length | Combine each two |       | Native Subdataset |              | Random Partition |       |
|---------|--------------|------------------|-------|-------------------|--------------|------------------|-------|
|         |              | NLL              | MAPE  | NLL               | MAPE         | NLL              | MAPE  |
| ETTh1   | 96           | 1.837            | 0.782 | <b>1.760</b>      | <b>0.694</b> | 1.965            | 0.752 |
|         | 192          | 1.860            | 0.740 | <b>1.766</b>      | <b>0.682</b> | 1.969            | 0.723 |
|         | 336          | 1.885            | 0.834 | <b>1.825</b>      | <b>0.746</b> | 1.998            | 0.832 |
|         | 720          | 1.911            | 0.991 | <b>1.853</b>      | <b>0.947</b> | 2.009            | 0.934 |

We observe two key findings that align with our initial assumptions. First, we assume that the data exhibits locality within partitions based on semantic similarity (e.g., time series domains); consequently, random partitioning yields the poorest performance due to a lack of semantic coherence. Second, coarser granularity weakens the locality within each partition, thereby degrading performance relative to the “Native Subdataset” configuration.

## D.3 Sensitivity to decay factor $\beta$

The performance in Table 10 is not very sensitive to the  $\beta$  setting as long as they are set to a reasonable range. In our experiment throughout the paper, we choose  $\beta = 0.01$ .

## D.4 Sensitivity to SNR filter bar

We also conducted an experiment to evaluate the impact of the SNR threshold on performance in Table 11. This filtering mechanism is designed to eliminate noisy data points based on domain

**Table 10: Performance comparison of different beta values.**

| Dataset | Pred. length | $\beta=0.1$ |       | $\beta=0.01$ |              | $\beta=0.001$ |       | $\beta=0.0001$ |       |
|---------|--------------|-------------|-------|--------------|--------------|---------------|-------|----------------|-------|
|         |              | NLL         | MAPE  | NLL          | MAPE         | NLL           | MAPE  | NLL            | MAPE  |
| ETTh1   | 96           | 1.744       | 0.814 | 1.760        | <b>0.694</b> | <b>1.688</b>  | 0.758 | 1.762          | 0.739 |
|         | 192          | 1.729       | 0.727 | 1.766        | <b>0.682</b> | <b>1.717</b>  | 0.714 | 1.754          | 0.704 |
|         | 336          | 1.774       | 0.835 | 1.825        | <b>0.746</b> | <b>1.753</b>  | 0.783 | 1.781          | 0.787 |
|         | 720          | 1.807       | 1.061 | 1.853        | <b>0.947</b> | <b>1.783</b>  | 0.962 | 1.793          | 0.953 |

knowledge, thus complementing the influence score-based filtering. We observe that an overly strict threshold (e.g., 5dB) leads to the exclusion of excessive data, which degrades performance. Optimal results are achieved at relatively small values of  $k$ . While more sophisticated hyperparameter tuning strategies may exist, we leave such exploration for future work and adopt  $k = 3\text{dB}$  for the remainder of this paper.

**Table 11: Performance comparison of different SNR values.**

| Dataset | Pred. length | k=1dB |       | k=3dB        |              | k=5dB |       |
|---------|--------------|-------|-------|--------------|--------------|-------|-------|
|         |              | NLL   | MAPE  | NLL          | MAPE         | NLL   | MAPE  |
| ETTh1   | 96           | 1.630 | 0.647 | <b>1.557</b> | <b>0.549</b> | 1.672 | 0.633 |
|         | 192          | 1.709 | 0.764 | <b>1.627</b> | <b>0.672</b> | 1.753 | 0.800 |
|         | 336          | 1.751 | 0.734 | <b>1.658</b> | <b>0.641</b> | 1.769 | 0.746 |
|         | 720          | 1.785 | 0.762 | <b>1.690</b> | <b>0.646</b> | 1.775 | 0.733 |

## D.5 Sensitivity to validation set selection

Table 12 presents the results for various validation set sizes. Throughout this paper, we use a fixed validation size of 32.

**Table 12: Performance comparison of different batch sizes.**

| Dataset | Pred. length | Val size=8 |       | Val size=32 |              | Val size=128 |              |
|---------|--------------|------------|-------|-------------|--------------|--------------|--------------|
|         |              | NLL        | MAPE  | NLL         | MAPE         | NLL          | MAPE         |
| ETTh1   | 96           | 1.633      | 0.515 | 1.557       | 0.549        | <b>1.506</b> | <b>0.505</b> |
|         | 192          | 1.730      | 0.728 | 1.627       | 0.672        | <b>1.569</b> | <b>0.584</b> |
|         | 336          | 1.787      | 0.758 | 1.658       | 0.641        | <b>1.644</b> | <b>0.587</b> |
|         | 720          | 1.825      | 0.807 | 1.690       | <b>0.646</b> | <b>1.710</b> | 0.672        |